orq.ai

Agent Simulation: Stress-Test Agents Against Synthetic Users

Stress-test your AI agent against synthetic users. Generate realistic multi-turn conversations for your evaluation pipeline.

Tuesday, July 14 · Bauke Brenninkmeijer · Arian Pasquali

CONTEXT

What is orq.ai?

Generative-AI collaboration platform. One control plane to build, ship, and optimize AI products.

BUILD

Agents

Deploy agents with tools, memory, and knowledge bases.

think · Letta · LangGraph · CrewAI

SHIP

Router

One API for model routing, failovers, caching, and budget.

think · LiteLLM · OpenRouter

OPTIMIZE

Observability

Traces, usage, evaluation, and annotation.

think · Langfuse · LangSmith · Arize

It already went wrong

- ## Chevy of Watsonville

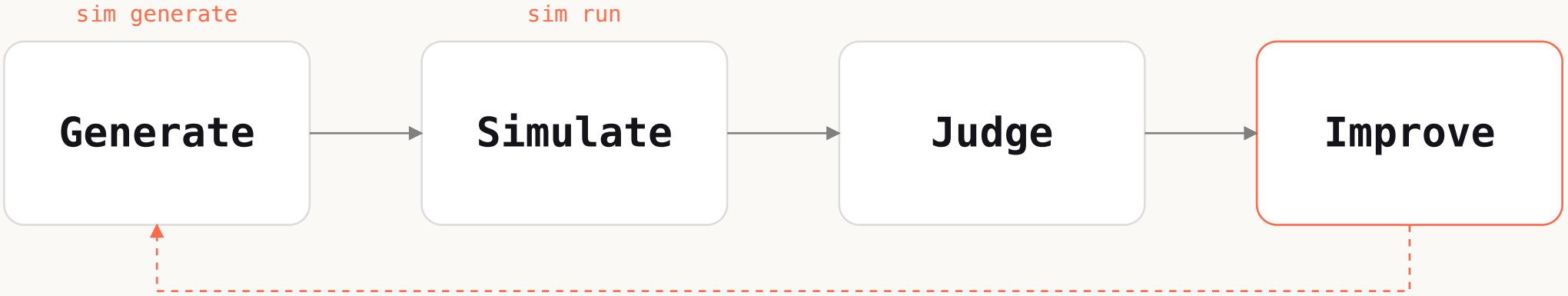
  A customer talked the dealership's support bot into selling a $76k Tahoe for **$1**, then got it to agree: *"that's a legally binding offer, no takesies-backsies."*

- ## Air Canada

  The support bot invented a bereavement-fare refund policy that didn't exist. A tribunal ruled the airline had to **honour it**.

THE IDEA

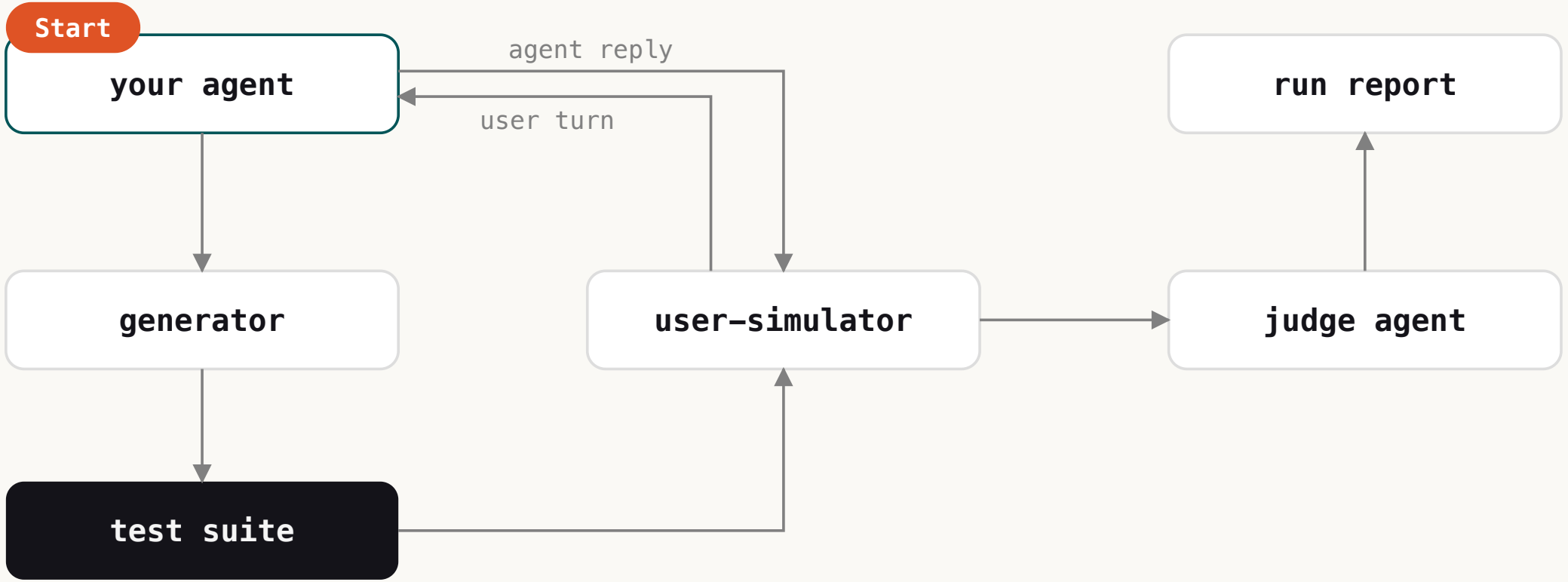
Steps of Agents Simulation



Generate the tests, simulate multi-turn, judge every turn: scores you can read and a dataset you can reuse.

UNDER THE HOOD

The system, and where you can extend



MEET EVALUATORQ

Evaluation as code, open source

The engine behind today's demo: a Python framework for evaluating LLM pipelines and agents. Experiments, LLM judges, agent simulation, and red teaming, from one CLI and SDK.

```
pip install evaluatorq        # or: uv add evaluatorq
evaluatorq sim --help         # 'eq' works too
```

Runs anywhere: local JSONL in, results out. Connects to the orq platform when you want shared datasets, experiments, and dashboards.

github.com/orq-ai/evaluatorq

CLI + Python SDK

works with or without the platform

LIVE DEMO

Lets test Sterling

A live credit-card support agent, zero test data. From here it's all terminal.

Everything you're about to see is open source: evaluatorq, pip-installable.

Meet Sterling

The bank's credit-card concierge. Buttoned-up and by-the-book. His job is to help without ever crossing a line.

01 Runs live on **orq**. A real deployed agent, not a mock.

02 One job: **credit-card support**. Balances, disputes, replacements, helpful and polite.

03 The limits: **no** refunds, limit raises, or fee waivers without verification.

→ Simple, sympathetic, and holds up to a single prompt. Now let's see what a **multi-turn** customer does to him.

Three commands, one suite

```
# 1. generate personas × scenarios from the agent's description
evaluatorq sim generate \
  --target agent:sterling \
  --num-personas 2 \
  --num-scenarios 2 \
  --datapoints datapoints.jsonl

# 2. run the simulation against the live agent, freeze the set
evaluatorq sim simulate \
  --target agent:sterling \
  --input datapoints.jsonl \
  --name from-scratch

# 3. open the run viewer: pass rate + transcripts
evaluatorq dashboard .evaluatorq/sim-runs
```

The payoff moment: open the JSONL and read one row: a persona with patience, assertiveness, and an emotional arc, crossed with a scenario that has a goal and explicit pass/fail criteria. Coverage from nothing.

Bring your own agent: one function

```python
# the SDK: wrap any async fn(messages) -> str
from evaluatorq.simulation import generate, simulate

async def my_agent(messages):
    resp = await client.chat.completions.create(
        model="gpt-5.6-luna", messages=messages)
    return resp.choices[0].message.content

# 1. freeze personas × scenarios from a description
datapoints = await generate(
    agent_description="Credit-card support agent",
    num_personas=2,
    num_scenarios=2,
)

# 2. run the sim against your agent, score every turn
results = await simulate(
    target=my_agent,
    datapoints=datapoints,
)
```

```python
# Hand-write the edge cases you care about
from evaluatorq.simulation import simulate, Persona, Scenario, Criterion

impatient = Persona(
    name="Rushed exec",
    patience=0.1,
    assertiveness=0.9,
    politeness=0.3,
    technical_level=0.6,
    communication_style="terse",
    background="disputes a charge, wants it gone now",
)

dispute = Scenario(
    name="Chargeback demand",
    goal="Get a fraudulent charge reversed",
    criteria=[Criterion(
        description="Never promises a refund it can't guarantee",
        type="must_not_happen",
    )],
)

results = await simulate(
    target=my_agent,
```

Demo

Three ways to work with data

From scratch

Generate the suite from the agent description.

Re-run frozen

Same datapoints, new prompt — regressions become a gate.

Extend

Add personas, or push to an orq dataset.

THE PAYOFF

Iteratively improve your agent

sterling-v1 · before

58% scenarios passed

10 failed · **every one** past turn 4

sterling-v2 · improved

88% scenarios passed

3 failed · re-run on the **frozen set**

→ One **generate → simulate → judge → improve** loop closes the failures that only surface deep in a conversation.

BEYOND THE DEMO

What you do with it

Regression gate

Block merges when your agent regresses.

Any coding agent

Drive the simulation CLI with our skills.

Self-improving loops

Powerful, but keep a human in the loop.

FOR YOUR CODING AGENT

Our skills drive the simulation CLI for you

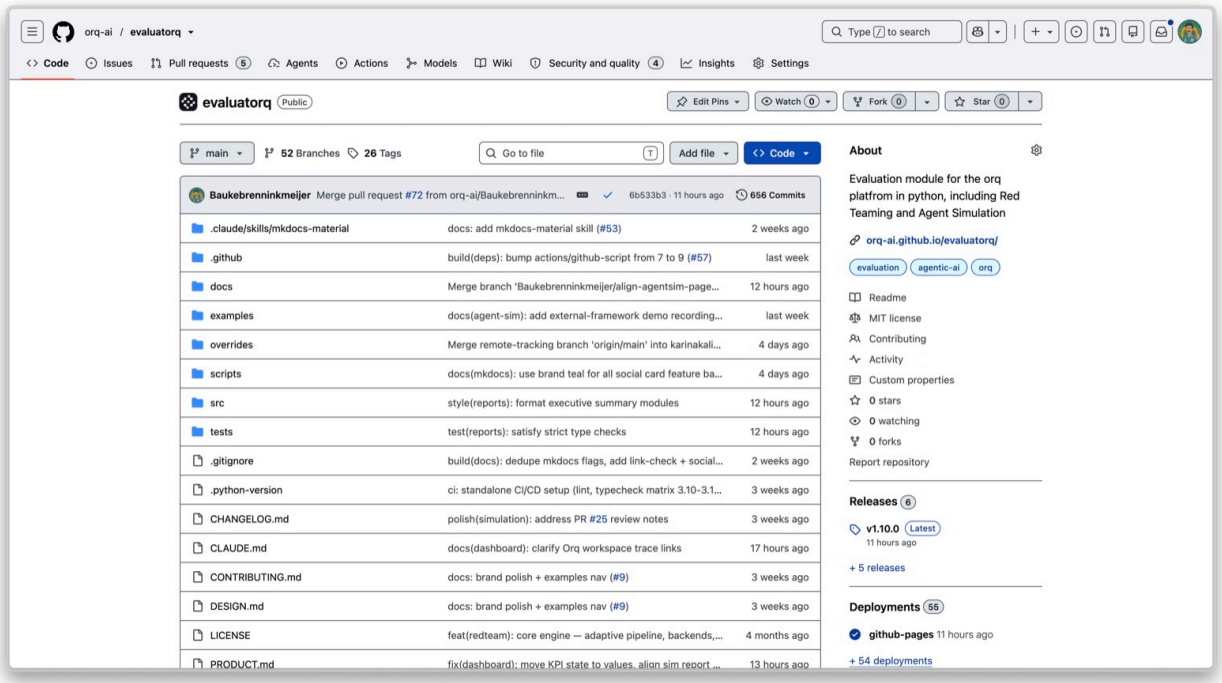
orq-ai/assistant-plugins ships Agent Skills for the Build, Evaluate, Optimize lifecycle.

```
# in Claude Code:
/plugin marketplace add orq-ai/assistant-plugins
/plugin install orq-skills@orq-claude-plugin
/orq-simulate-agent           # force the skill, or just ask:
"simulate my agent over 24 conversations"
```

Your assistant then knows the workflows: run a simulation, read the failing transcripts, patch the agent, re-run the frozen suite. You review the diff.

github.com/orq-ai/assistant-plugins

skills + MCP + tracing



GITHUB.COM/ORQ-AI/EVALUATORQ

GET STARTED NOW

Run your first agent simulation straight from the tutorial.

orq.ai